DevOps Troubleshooting — Part 4

Jira / ITSM Workflow

1. You receive a new P2 production incident ticket. What will you do first?
Answer:
First, I understand the business impact, affected application, number of users, and current symptoms from the ticket. Then I validate the issue using monitoring, logs, dashboards, and application checks.
I acknowledge the ticket, start investigation, and keep the ticket updated with important findings and actions.
Simple ga gurthupettuko:
P2 → impact understand → validate → investigate → ticket update.
Eng: First I understand the impact, validate the issue, start troubleshooting, and keep the incident ticket updated.
Example:
Jira → Incident ticket → Priority/P2 → Impact and affected service check cheyyi.
Monitoring → alerts and dashboards check cheyyi.
Application logs → error identify cheyyi.
Ticket lo → issue summary + investigation findings update cheyyi.
Issue critical ayithe → on-call/team escalation initiate cheyyi.

---

2. What is the difference between an Incident and a Service Request?
Answer:
An Incident is an unplanned interruption or degradation of a service that needs restoration. A Service Request is a standard request such as access, software installation, environment creation, or information.
I classify the ticket correctly because the workflow, priority, SLA, and approval process can be different.
Simple ga gurthupettuko:
Incident = service problem. Service Request = standard user request.
Eng: An incident restores a disrupted service, while a service request handles a standard user requirement.
Example:
Production API down → **Incident**.
User asking for AWS access → **Service Request**.
Application failure → Incident workflow follow cheyyi.
Access request → approval + fulfillment workflow follow cheyyi.

---

3. What is a Change ticket and when would you create one?
Answer:
A Change ticket is used when I plan to modify a production system or infrastructure in a controlled manner. It normally contains the implementation plan, risk, impact, validation steps, rollback plan, and required approvals.
I create and follow the change process before making a planned production modification.
Simple ga gurthupettuko:
Production modification → Change ticket → plan → risk → approval → implementation → validation.
Eng: A change ticket provides controlled documentation and approval for planned production modifications.
Example:
Jira/ITSM → Change ticket create cheyyi.
Implementation steps add cheyyi.
Risk and impact mention cheyyi.
Rollback plan add cheyyi.
Required approval obtain cheyyi.
Change window lo implementation chesi → validation results update cheyyi.

---

4. What is a Problem ticket in ITSM?
Answer:
A Problem ticket is used to investigate the underlying root cause of one or more recurring incidents. An Incident focuses on restoring the service, while a Problem focuses on preventing the issue from happening again.
I document the root cause, corrective action, and preventive action in the Problem record.
Simple ga gurthupettuko:
Incident = restore service. Problem = find root cause and prevent repeat.
Eng: An incident restores service, while a problem investigates the underlying cause and prevents recurrence.
Example:
Same application crash repeatedly → related incidents identify cheyyi.
Problem ticket create cheyyi.
Logs/metrics/history analyze cheyyi.
Root cause identify cheyyi.
Permanent fix plan create cheyyi.
Fix implement chesi → recurrence monitor cheyyi.

5. How do you decide whether an incident is P1 or P2?
Answer:
I consider business impact and urgency rather than only the technical severity. A complete production outage affecting many customers may be P1, while a serious issue affecting a limited service or user group may be P2.
I follow the organization's incident priority matrix and escalation policy.
Simple ga gurthupettuko:
Priority = impact + urgency + business criticality.
Eng: I classify priority based on business impact and urgency according to the organization's priority matrix.
Example:
Entire production unavailable → likely **P1**.
Critical feature unavailable for limited users → may be **P2**.
Jira → Priority matrix check cheyyi.
Business impact document cheyyi.
Correct priority assign chesi → required escalation start cheyyi.

---

6. An incident is approaching its SLA deadline. What will you do?
Answer:
First, I check the current incident status and remaining SLA time. If the issue is not resolved, I escalate to the appropriate team and clearly communicate the business impact and current blocker.
I continue updating the ticket so that the investigation and escalation history are documented.
Simple ga gurthupettuko:
SLA near → status check → escalate → communicate → ticket update.
Eng: I escalate early when an SLA is at risk and keep the ticket updated with the current status and blockers.
Example:
Jira → SLA timer check cheyyi.
Current investigation status update cheyyi.
Blocking team identify cheyyi.
On-call/lead ki escalation cheyyi.
Ticket lo → blocker + escalation + next action update cheyyi.

---

7. You cannot resolve a production incident within your team. How will you escalate it?
Answer:
I first document what has already been checked, the evidence collected, the current impact, and the suspected cause. Then I escalate to the relevant SME, application team, network team, cloud team, or vendor depending on the issue.
I avoid simply saying "issue is not ours" and instead provide enough technical information for the next team to continue quickly.
Simple ga gurthupettuko:
Cannot solve → evidence collect → correct team → proper handoff → continue tracking.
Eng: I escalate with evidence and troubleshooting history so the next team can continue without repeating the same checks.
Example:
Jira → troubleshooting steps update cheyyi.
Logs/error screenshots or relevant output attach cheyyi.
Impact and affected service mention cheyyi.
Correct resolver team assign cheyyi.
Escalation comment add chesi → ticket follow-up continue cheyyi.

---

8. What information should you add while updating a production incident ticket?
Answer:
I include the current impact, symptoms, investigation performed, findings, actions taken, current status, blockers, and next steps.
I avoid vague updates such as "checking" and provide useful information that another engineer can understand.
Simple ga gurthupettuko:
Update = impact + what checked + finding + action + status + next step.
Eng: I keep ticket updates concise but technical enough for another engineer to understand the current situation.
Example:
Bad update → "We are checking."
Good update → "API 5xx increased after deployment. Rollback completed and error rate is decreasing. Application team is validating remaining failures."
Jira → timestamped updates add cheyyi.
Next action and owner mention cheyyi.

---

9. When should you close an incident ticket?
Answer:
I close an incident only after confirming that the service has recovered and the required validation is completed. I also document the resolution, root cause if known, and any follow-up Problem or Change ticket.
If the organization requires user confirmation, I wait for that confirmation before closure.
Simple ga gurthupettuko:
Fix → validate → resolution document → follow-up tickets → close.
Eng: I close the incident only after service recovery is validated and the resolution is properly documented.
Example:
Monitoring → errors normal ayyaya check cheyyi.
Application → functional test cheyyi.
Users/business → required confirmation obtain cheyyi.

Jira → resolution details update cheyyi.
Related Problem/Change link cheyyi.
Ticket → Resolved/Closed according to workflow.

---

10. Incident is resolved, but the root cause is still unknown. Will you close it?
Answer:
If the service is fully restored, the Incident can usually be resolved according to the organization's process, but I create or link a Problem ticket for root-cause investigation if required.
I make sure the unresolved root cause is not forgotten and assign appropriate follow-up ownership.
Simple ga gurthupettuko:
Service restored but RCA pending → Incident resolve → Problem ticket → RCA continue.
Eng: The incident can be resolved after service recovery, while a separate Problem process can continue the root-cause investigation.
Example:
Production service → recovered verify cheyyi.
Jira Incident → resolution update cheyyi.
Problem ticket → RCA investigation create/link cheyyi.
Owner assign cheyyi.
Incident → required workflow prakaram resolve/close cheyyi.

Production Support / On-Call
11. You receive a production alert during on-call. What is your first response?
Answer:
First, I acknowledge the alert and determine whether it represents a real customer-impacting issue. Then I check the affected service, severity, recent changes, monitoring dashboards, logs, and current health.
If required, I immediately escalate based on the incident severity.
Simple ga gurthupettuko:
Alert → acknowledge → impact → severity → investigate → escalate if required.
Eng: I acknowledge the alert, validate the impact, identify severity, and start troubleshooting immediately.
Example:
Monitoring → alert details check cheyyi.
Dashboard → affected service health chudu.
Recent deployment → change check cheyyi.
Logs → errors identify cheyyi.
P1/P2 ayithe → incident process + escalation start cheyyi.

---

12. During a P1 incident, what is an incident bridge?
Answer:
An incident bridge is a dedicated communication channel or conference used by multiple teams to coordinate during a major incident.
It helps teams share findings, assign actions, avoid duplicate work, and keep everyone aligned on the current situation.
Simple ga gurthupettuko:
P1 → bridge → teams together → actions assign → updates coordinate.
Eng: An incident bridge provides one place for teams to coordinate troubleshooting and recovery during a major incident.
Example:
P1 declared → incident bridge open cheyyi.
Incident commander/lead identify cheyyi.
Teams join → DevOps/App/Network/Cloud as required.
Each team ki action assign cheyyi.
Findings bridge lo communicate cheyyi.
Recovery verify chesi → bridge close cheyyi.

---

13. During a production incident, business stakeholders are asking for updates every few minutes. How will you handle it?
Answer:
I provide concise updates focused on impact, current status, actions being taken, and the next expected update.
I avoid sharing unconfirmed root causes and technical details that can create confusion.
Simple ga gurthupettuko:
Stakeholder update → impact + status + action + next update.
Eng: I communicate confirmed facts, business impact, current actions, and the next update time without speculating.
Example:
“Production API is degraded and some requests are failing. Engineering teams are investigating the backend dependency. Next update in 15 minutes.”
Incident bridge → technical troubleshooting continue cheyyi.
Stakeholders → concise status updates provide cheyyi.

14. Two teams disagree about the root cause during a P1 incident. What will you do?
Answer:
I avoid spending too much time debating assumptions. I ask both teams to provide evidence and focus on tests that can quickly confirm or eliminate each hypothesis.
The immediate priority is service restoration; deeper root-cause analysis can continue after recovery.
Simple ga gurthupettuko:
Root cause disagreement → evidence → test hypotheses → restore service first → RCA later.
Eng: I use evidence-driven troubleshooting and prioritize service recovery over arguing about unproven root causes.
Example:
Team A → application issue hypothesis.
Team B → network issue hypothesis.
Logs/metrics/traces → evidence collect cheyyi.
Controlled test → hypothesis validate cheyyi.
Service restore first.
After recovery → detailed RCA document cheyyi.

---

15. What should you include in an on-call handover?
Answer:
I include the current incident status, impact, troubleshooting already performed, findings, actions completed, pending actions, owners, risks, and next steps.
The incoming engineer should be able to continue without restarting the investigation from the beginning.
Simple ga gurthupettuko:
Handover = impact + status + checked + findings + pending + owner + next step.
Eng: A good handover gives the next engineer enough context to continue immediately.
Example:
Incident → current impact mention cheyyi.
Completed checks → document cheyyi.
Known issue/root cause → mention cheyyi.
Pending action → owner assign cheyyi.
Monitoring status → current condition add cheyyi.
Jira → all details update chesi → next on-call engineer ki handover cheyyi.

---

16. A production deployment caused errors. What will you do?
Answer:
First, I confirm the correlation between the deployment and the errors using timestamps, monitoring, logs, and deployment history.
If customer impact is significant, I follow the rollback or mitigation procedure.
After recovery, I investigate why the deployment caused the issue and document the incident and corrective actions.
Simple ga gurthupettuko:
Deploy → errors → correlate → impact → rollback/mitigate → recover → RCA.
Eng: I confirm whether the deployment caused the issue, restore service quickly, and then investigate the deployment failure.
Example:
CI/CD → deployment timestamp check cheyyi.
Grafana/Prometheus → before/after metrics compare cheyyi.
Application logs → new errors check cheyyi.
If rollback approved → previous version deploy cheyyi.
Health checks → recovery verify cheyyi.
Incident + Change/Problem records update cheyyi.

---

17. Production is partially degraded, but not completely down. How will you handle it?
Answer:
I determine the exact scope of degradation, affected functionality, and customer impact. Then I prioritize mitigation to reduce the impact while continuing root-cause investigation.
I communicate the partial outage clearly and monitor whether the impact is increasing or decreasing.
Simple ga gurthupettuko:
Partial outage → scope → impact → mitigate → investigate → monitor.
Eng: I focus on reducing customer impact while continuing investigation into the underlying cause.
Example:
Monitoring → error rate/latency check cheyyi.
Affected endpoint → identify cheyyi.
Load balancer → unhealthy targets check cheyyi.
Traffic reduction/routing/rollback → approved mitigation apply cheyyi.
Metrics → degradation improve ayyindo verify cheyyi.

---

18. What is an RCA and what should it contain?
Answer:
RCA means Root Cause Analysis. It explains what happened, when it happened, customer impact, the technical root cause, contributing factors, recovery actions, and preventive actions.
A good RCA focuses on learning and prevention rather than blaming individuals.
Simple ga gurthupettuko:
RCA = what happened + impact + root cause + recovery + prevention.
Eng: RCA explains the incident and defines actions to prevent the same failure from recurring.

Example:
Incident timeline → timestamps document cheyyi.
Impact → affected services/users mention cheyyi.
Root cause → technical cause identify cheyyi.
Recovery → what restored service explain cheyyi.
Corrective action → permanent fix add cheyyi.
Preventive action → monitoring/process improvement add cheyyi.

---

19. A P1 incident is resolved. What checks will you perform before declaring it fully recovered?
Answer:
I verify application health, error rates, latency, infrastructure health, dependency status, and critical user journeys.
I also monitor the system for a reasonable period to ensure the issue does not immediately return.
Simple ga gurthupettuko:
P1 fixed → app health + errors + latency + dependencies + user journey → monitor → close.
Eng: I validate recovery across technical and user-facing signals before declaring the incident resolved.
Example:
Prometheus → error rate normal aa check cheyyi.
Grafana → latency/traffic dashboards check cheyyi.
Application → health endpoint test cheyyi.
ALB/Nginx → healthy targets verify cheyyi.
Critical user flow → functional test cheyyi.
Monitoring → issue recurring ledu ani confirm cheyyi.

---

20. You made a production change during an emergency. What will you document afterward?
Answer:
I document what was changed, why it was necessary, who approved or authorized it, the implementation time, commands or configuration changed, validation results, and rollback information.
I also ensure the emergency change is linked to the relevant incident and followed up through the normal change process if required.
Simple ga gurthupettuko:
Emergency change → what + why + who + when + validation + rollback → document/link.
Eng: I document the emergency change completely and link it to the incident and follow-up change process.
Example:
Jira Incident → emergency action details update cheyyi.
Change record → change details add cheyyi.
Commands/config changes → document cheyyi.
Validation → service recovered evidence add cheyyi.
Rollback → required steps document cheyyi.
Post-incident review → permanent fix identify cheyyi.

GitLab CI/CD
21. GitLab pipeline is stuck in Pending state. What will you check?
Answer:
First, I check whether a GitLab Runner is available and whether the job's tags match any available runner. Then I check runner status, concurrency limits, protected runner settings, and whether runners are overloaded.
I fix the runner availability or tagging issue and rerun the pipeline.
Simple ga gurthupettuko:
Pending → Runner available aa → tags match aa → runner busy/protected aa → fix → rerun.
Eng: I check runner availability, job tags, runner capacity, and protected-runner configuration.
Example:
GitLab → Project → Settings → CI/CD → Runners → runner status chudu.
.gitlab-ci.yml → tags: inspect cheyyi.
Runner tags → job tags tho match avuthunnaya check cheyyi.
Runner host → service status verify cheyyi.
Runner capacity/concurrency check chesi → pipeline rerun cheyyi.

---

22. GitLab pipeline fails because no runner matches the job tags. What will you do?
Answer:
I compare the tags defined in the job with the tags assigned to available runners. If the job requires a specific runner, I verify that the runner is online and available to the project.
I correct the tags or runner assignment according to the intended architecture.
Simple ga gurthupettuko:
No matching runner → job tags vs runner tags compare → runner availability → correct assignment.
Eng: I ensure the job's tags match an available runner that is authorized to execute the project.
Example:
.gitlab-ci.yml → job tags check cheyyi.
GitLab → Settings → CI/CD → Runners → runner tags chudu.
Runner online aa verify cheyyi.
Project access/runner assignment check cheyyi.
Tag mismatch ayithe → correct tag update chesi → pipeline rerun cheyyi.

23. GitLab pipeline suddenly fails with "command not found." What will you check?
Answer:
First, I identify which command is missing and check the runner environment or Docker image used by the job. Then I verify whether the required package/tool was previously available and whether the image or runner changed.
I add the dependency properly to the build image or installation step rather than relying on an undocumented runner dependency.
Simple ga gurthupettuko:
Command not found → runner/image → tool installed aa → recent image change → dependency add → rerun.
Eng: I verify the runner environment and make required dependencies explicit in the pipeline.
Example:
GitLab job log → missing command identify cheyyi.
.gitlab-ci.yml → image: inspect cheyyi.
which <command> → tool available aa check cheyyi.
<command> --version → version verify cheyyi.
Required package install/image update chesi → pipeline rerun cheyyi.

---

24. GitLab pipeline is successful, but the artifact is missing. What will you check?
Answer:
First, I check the job's artifact configuration and whether the expected files were actually generated. Then I verify artifact paths, relative paths, expiration settings, and whether the correct job produced the artifact.
I correct the artifact configuration and rerun the job.
Simple ga gurthupettuko:
Pipeline success but artifact missing → file generated aa → paths → artifacts config → expiry → rerun.
Eng: I verify that the file exists and that the GitLab artifact configuration points to the correct path.
Example:
Job log → build output check cheyyi.
ls -lah <build-directory> → artifact file exists aa chudu.
.gitlab-ci.yml → artifacts: section inspect cheyyi.
paths: correct aa verify cheyyi.
expire_in: check cheyyi.
Config fix chesi → pipeline rerun chesi → artifact download verify cheyyi.

---

25. GitLab pipeline is failing only on the main branch, while feature branches pass. What will you compare?
Answer:
First, I compare the branch-specific configuration, variables, protected variables, rules, dependencies, and recent commits.
I also check whether the main branch uses different deployment permissions, runners, environments, or secrets.
Simple ga gurthupettuko:
Main fail, feature pass → config/variables/rules/secrets/runner/permissions compare.
Eng: I compare branch-specific configuration and protected resources between the successful and failing pipelines.
Example:
GitLab → Pipeline → successful feature pipeline vs main pipeline compare cheyyi.
.gitlab-ci.yml → rules/only/except inspect cheyyi.
CI/CD Variables → protected variables check cheyyi.
Runner tags → compare cheyyi.
Recent commits → git log compare cheyyi.
Difference identify chesi → fix → main pipeline rerun cheyyi.

---

26. GitLab deployment job fails because the CI variable is empty. What will you check?
Answer:
First, I verify whether the variable exists and whether it is available to that project, branch, environment, or runner. Then I check protected and masked variable settings and variable scope.
I also verify that the variable name used in .gitlab-ci.yml exactly matches the configured variable.
Simple ga gurthupettuko:
Variable empty → variable exists aa → protected/scope → name match → environment check.
Eng: I check variable existence, scope, protection settings, and the exact variable name used by the job.
Example:
GitLab → Settings → CI/CD → Variables → variable exists aa chudu.
Protected setting → main/protected branch tho match aa check cheyyi.
Environment scope → correct environment ki available aa verify cheyyi.
.gitlab-ci.yml → $VARIABLE_NAME spelling check cheyyi.
Correct config tarvatha → deployment rerun cheyyi.

---

27. GitLab job works on one runner but fails on another. What will you compare?
Answer:
I compare the runner OS, Docker image, installed tools, environment variables, permissions, network access, and runner configuration.
The goal is to identify an environment difference rather than changing the application unnecessarily.
Simple ga gurthupettuko:
One runner works, another fails → OS/image/tools/env/permissions/network compare.
Eng: I compare the runner environments to identify the environmental difference causing the failure.
Example:
Runner 1 → uname -a and tool versions check cheyyi.

Runner 2 → same commands run cheyyi.
docker --version / java -version / node --version → versions compare cheyyi.
Environment variables and permissions compare cheyyi.
Network access verify cheyyi.
Difference fix chesi → job rerun cheyyi.

****28. GitLab pipeline takes much longer than before. What will you investigate?**
Answer:
First, I compare the current pipeline duration with the last successful or fast pipeline. Then I identify which stage became slower and check dependency downloads, runner performance, cache behavior, test execution, artifact size, and recent configuration changes.
I optimize the slow stage rather than increasing runner resources without evidence.
Simple ga gurthupettuko:
Pipeline slow → stage identify → old vs new compare → runner/cache/dependency/test/artifact check.
Eng: I identify the stage causing the slowdown and compare it with the previous working pipeline.
Example:
GitLab → Pipeline → job duration compare cheyyi.
Slow job → job log inspect cheyyi.
Runner → CPU/memory/disk check cheyyi.
Cache → cache hit/miss behavior check cheyyi.
Dependency download time compare cheyyi.
Slow stage optimize chesi → pipeline duration verify cheyyi.

****29. GitLab deployment succeeds, but the application is not updated. What will you check?**
Answer:
First, I verify whether the deployment job actually deployed the intended version and environment. Then I check image/tag/version, deployment target, rollout status, and whether an older instance is still serving traffic.
I validate the running application version from the environment rather than relying only on a successful CI job.
Simple ga gurthupettuko:
Deploy success but old app → intended version → target environment → rollout → running version → traffic.
Eng: I verify the deployed version in the actual environment and confirm that traffic is reaching the updated instances.
Example:
GitLab job log → deployed image/version check cheyyi.
Kubernetes → kubectl get pods -n <ns> chudu.
kubectl describe deployment <name> -n <ns> → image/version verify cheyyi.
kubectl rollout status deployment/<name> -n <ns> → rollout check cheyyi.
Application endpoint → version response verify cheyyi.
Old pods/traffic issue unte → fix chesi → version verify cheyyi.

****30. GitLab pipeline fails after changing** .gitlab-ci.yml**. What will you do?**
Answer:
First, I validate the CI configuration syntax and compare the changed file with the previous working version. Then I identify whether the failure is caused by YAML structure, job dependencies, rules, variables, stages, scripts, or runner requirements.
I make the smallest required correction and rerun the pipeline.
Simple ga gurthupettuko:
CI file change → validate syntax → compare old/new → identify config issue → fix → rerun.
Eng: I validate the GitLab CI configuration and compare it with the last working version before making further changes.
Example:
GitLab → CI/CD → Pipeline Editor → Validate → YAML validation cheyyi.
.gitlab-ci.yml → changed lines inspect cheyyi.
git diff <old-commit> <new-commit> -- .gitlab-ci.yml → changes identify cheyyi.
Stages/dependencies/rules/variables verify cheyyi.
Fix chesi → pipeline rerun → all required jobs successful ga complete ayyayo verify che